期末项目 · 代码拆解

十段代码，
十次「不把正确性交给模型」

外刊精读教练（ReadCoach）· 每段都标了真实文件与行号，都回答三个问题：
为什么这么写、逻辑怎么走、用到哪些知识。

配置层 ×1　编排层 ×2　工具层 ×6　界面层 ×1

下方箭头或滚轮翻页 · 按 F 全屏 · 共 13 页

地图

十段代码分布在四层，一条调用链串起来

用户一句话 → app.py 界面 → build_system_prompt → StateGraph 循环 → 七个工具 → outputs/ 文件

配置层

写盘权限边界

① `safe_output_path()`
config.py:40

Agent 只能写 outputs/，越界直接报错。

编排层

循环与提示词

② `StateGraph`
graph.py:43
③ `build_system_prompt()`
prompts.py:126

agent ⇄ tools 循环；技能按需注入。

工具层

接口、数据、校验

④ `@tool + docstring`
dictionary.py:22
⑤ `load_lexicon()`
lexicon.py:17
⑥ `_verify()`
wordlist.py:68
⑦ `_drop_source_field()`
wordlist.py:35
⑧ `_word_pattern()`
worksheet.py:22
⑨ `record_mistakes()`
memory.py:135

七个工具共用一份词库、一套校验。

界面层 ⑩ `graph.stream()` 事件循环 app.py:105 —— 把每一步摊给用户看

代码 01 • config.py:40–72 • 配置层

文件名是模型给的，所以它不能被信任

```python
def safe_output_path(filename: str, default_ext: str = ".txt") -> Path:
    raw = (filename or "").strip().replace("\\", "/")
    # 模型常写 outputs/xxx.csv，先剥掉前缀再校验
    if raw.lower().startswith("outputs/"):
        raw = raw[len("outputs/"):]

    if "/" in raw or raw in (".", ".."):
        raise ValueError(f"拒绝写入 outputs 之外的路径：{filename}")

    name = "".join(c for c in raw if c not in _ILLEGAL).strip()
    if not os.path.splitext(name)[1]:
        name += default_ext

    target = (OUTPUT_DIR / name).resolve()
    if target.parent != OUTPUT_DIR.resolve():
        raise ValueError(f"拒绝写入 outputs 之外的路径：{filename}")
    return target
```

为什么这么写

这个 `filename` 是模型生成的，属于不可信输入。它可能给你 `../.env`。

为什么是报错，不是改名

初版用 `basename()` 把 `../evil.txt` 静默截成 `evil.txt`：文件安全了，但模型以为自己写对了，汇报路径就是错的。报错它才能改参数。

逻辑三步

剥前缀 → 拒绝任何目录层级 → `resolve()` 后复核父目录（防符号链接和 `..` 绕过）。

用到的知识

`pathlib` 的 `/` 拼接、`resolve()` 规范化路径、路径穿越、fail fast、不可信输入

一句话：安全的做法不止一种，但「静默成功」对 Agent 最糟——它拿不到反馈，就会一直错下去。

代码 02 • agent/graph.py:43–55 • 编排层

四行代码，就是一个 agent ⇄ tools 循环

```python
class AgentState(TypedDict):
    messages: Annotated[list, add_messages]

builder = StateGraph(AgentState)
builder.add_node("agent", agent_node)
builder.add_node("tools", ToolNode(ALL_TOOLS))

builder.add_edge(START, "agent")
builder.add_conditional_edges(
    "agent",
    tools_condition,                        # 有 tool_calls 吗?
    {"tools": "tools", END: END},
)
builder.add_edge("tools", "agent")          # 执行完回到 agent

return builder.compile()
```

为什么不用 while 循环

自己写 `while True` 要自己管退出条件，模型反复调工具会把进程跑死。框架自带递归上限和状态管理。

逻辑

agent 产出 `tool_calls` → 条件边判断有没有工具要调 → 有就走 tools 执行 → 回到 agent 看结果 → 没有就 END。

`add_messages` 是关键

它是 reducer：新消息追加到历史，不是覆盖。没有它，每轮只剩最后一条消息，模型会「失忆」

用到的知识

`TypedDict` 类型标注、`Annotated` + reducer、有向图与条件边、ReAct 范式、编译期校验。

一句话：这张图本身就长成汇报用的架构图——代码结构和讲解结构是同一个东西，不用另外画。

代码 03 • agent/prompts.py:89–113 • 编排层

技能不该常驻，命中触发词才加载

```python
SKILL_TRIGGERS = ("精读", "生词", "词表", "练习",
                  "外刊", "文章", "worksheet", "读一下")

def select_skill(query: str) -> str | None:
    if not query:
        return None
    lowered = query.lower()
    if not any(k in lowered for k in SKILL_TRIGGERS):
        return None                      # 没命中就完全不加载
    skill_file = SKILL_DIR / "intensive-reading" / "SKILL.md"
    if not skill_file.exists():
        return None
    return _strip_frontmatter(
        skill_file.read_text(encoding="utf-8")
    ).strip()

def build_system_prompt(query: str = "") -> str:
    exam = load_profile().get("target_exam", "CET-6")
    prompt = SYSTEM_PROMPT.format(exam=exam, profile=_profile_block())
    skill = select_skill(query)
    if skill:
        prompt += "\n\n---\n\n## 已加载技能：外刊精读\n\n" + skill
    return prompt
```

为什么分层

常驻人设要短、稳定（约 1.9k 字符）；技能流程要长、可替换。混在一起会撑爆上下文，改流程还得动主提示词。

逻辑

模板占位填档案 → 命中触发词才读 `SKILL.md` → 剥掉 frontmatter → 拼到末尾。

实测数字

相关请求注入 1717 字符；问「今天天气怎么样」完全不注入。**没命中的部分一个字符都不占。**

用到的知识

`str.format` 模板、按需加载、Markdown frontmatter 解析、上下文窗口有成本

一句话：加一个技能 = 加一个目录，主提示词一行不改。这就是 Skill 和系统提示词的本质区别。

代码 04 • tools/dictionary.py:22–30 • 工具层

docstring 不是注释，是写给模型的说明书

```python
from langchain_core.tools import tool

@tool
def lookup_word(word: str) -> str:
    """在本地词库中查一个英文单词，返回音标、词性、
    中文释义、例句和考纲等级。

    参数 word 只传单个英文单词，不要带句子或标点。
    如果返回 NOT_FOUND，说明该词不在本地词库中，
    你必须明确告诉用户「这个词不在本地词库内，
    释义来自模型推断，请自行核对」，不要假装查到了。
    """
    key = (word or "").strip().lower().strip(".,!?;:\"'()[]")
    if not key:
        return "ERROR: word 参数为空。"
    ...
```

装饰器做了什么

`@tool` 把普通函数包装成「模型可调用对象」：函数名变工具名，类型注解变参数 schema，docstring 变工具描述

为什么 docstring 要写这么啰嗦

模型就是靠这段文字决定「什么时候调、参数传什么、拿到 NOT_FOUND 该怎么说话」，所以接口文档即提示词

参数清洗

`strip(".,!?;:\"'()[]")` 去掉模型顺手带上的标点——它常把 `"emission,"` 整段传进来。

用到的知识

装饰器、类型注解 → JSON Schema、Function Calling 协议、`str.strip` 的字符集参数。

一句话：工具描述含糊，模型就乱传参数或干脆不调。写工具说明，本质上是在做提示词工程。

代码 05 • tools/lexicon.py:13–24 • 工具层

两个工具读同一份词库，判断标准只能有一个

```python
from functools import lru_cache

LEXICON_PATH = (Path(__file__).resolve().parent.parent
                / "data" / "lexicon.json")

@lru_cache(maxsize=1)
def load_lexicon() -> dict:
    with open(LEXICON_PATH, "r", encoding="utf-8") as f:
        return json.load(f)

def lookup(word: str) -> dict | None:
    key = (word or "").strip().lower().strip(".,!?;:\"'()[]")
    return load_lexicon().get(key) if key else None
```

为什么抽这一层

查词工具和词表工具读同一份 JSON。如果各写一遍判断逻辑，就会出现**「查词说没收录、词表却给了 CET-6」**——模型顺手就把幻觉补圆了。

逻辑

路径常量 + 缓存单例；两个工具都调 `lookup()`，「什么算已收录」只有一处定义

`lru_cache` 的作用

JSON 只读一次，后续命中内存缓存。主路径要查 8 次词，省 7 次文件 IO。

用到的知识

模块化、单一数据源 SSOT、记忆化缓存、`__file__` 定位相对路径（不依赖启动目录）

一句话：数据一致性靠的不是约定，是**只留一个入口**。两处各写一遍，早晚不一致。

代码 06 • tools/wordlist.py:68–83 • 工具层 • 全场最重要

模型编的考纲等级，用词库当场覆盖掉

```python
def _verify(rows: list[dict]):
    """拿词库复核每个词条的等级，返回
    (行，被纠正的词，未收录的词)。"""
    corrected, unverified = [], []
    for row in rows:
        known = lookup(row["word"])
        if known is None:
            # 查不到：不许猜，强制标未收录
            row["level"] = UNVERIFIED      # "未收录"
            row["source"] = "模型推断"
            unverified.append(row["word"])
        else:
            # 查得到：用词库等级覆盖模型填的
            if row["level"] and row["level"] != known["level"]:
                corrected.append(row["word"])
            row["level"] = known["level"]
            row["source"] = "本地词库"
    return rows, corrected, unverified
```

起因是实测

`structural`、`enforceable` 查词返回 NOT_FOUND，模型仍然给它们填了 CET-6 等级，写进 CSV。

为什么提示词拦不住

我先在提示词里加了一句「不许编造等级」，重跑之后它还是编，所以只能把校验写进代码。

两个返回值都有用

`corrected` / `unverified` 会回传给模型，它据此说出「含 2 个未收录词，建议核对权威词典」。

用到的知识

防御性编程、事实与生成分离、数据溯源（来源字段 → Word 标签）、返回值即约束

结论：**模型的自我约束不是约束。**校验放在代码里，不靠它自觉——这就是「工具」存在的意义。

代码 07 • tools/wordlist.py:25–63 • 工具层

对格式错的输入太宽容，等于默许错误

```
_SEP = re.compile(r"\s*[|｜\t]\s*")
# 模型会把「来源」也塞进 entries，变成六列，位置全错
_SOURCE_HINT = re.compile(
    r"(\.json$|\.db$|\.csv$|^本地词库$|^模型推断$|^未收录$)", re.I)

def _drop_source_field(parts: list[str]) -> list[str]:
    if len(parts) <= 5:
        return parts
    for i, p in enumerate(parts):
        if _SOURCE_HINT.search(p.strip()):
            return parts[:i] + parts[i + 1:]
    return parts[:5]

def _parse_entries(entries: str) -> list[dict]:
    ...
    parts = [p.strip() for p in _SEP.split(line) if p.strip()]
    parts = _drop_source_field(parts)          # 先对齐列
    if len(parts) < 2:
        continue
    ...
    "example": parts[4] if len(parts) > 4 else "",
```

这个 bug 是我自己引入的

我给工具返回值带过 `source`（来源）字段之后，模型下次就把 source 也塞进 `entries`：六列，例句被挤到第六位。

后果很隐蔽

被 `parts[:5]` 静默丢弃后，CSV 的 `example` 列全变成 `data/lexicon.json`。**不报错，但数据全错。**

踩坑记录：`match` vs `search`

初版用 `re.match` 匹配 `\.json$`，永远失败——`match` 只从开头比，必须改用 `search`，已加回归测试

用到的知识

正则锚点 `$` 与分组、`re.escape`、字段位置协议、静默截断的危害

一句话：工具的返回值会改变模型的下一次输入——加一列 `columns`，就可能多出六列 entries。这是闭环里必须防的地方。

代码 08 • tools/worksheet.py:22–44 • 工具层

原文里的 mitigating，也要能被挖空

```python
def _word_pattern(word: str) -> re.Pattern:
    w = word.strip().lower()
    forms = {w, w + "s", w + "es", w + "ed", w + "d", w + "ing"}
    if w.endswith("e"):
        # mitigate -> mitigating / mitigated
        forms |= {w[:-1] + "ing", w[:-1] + "ed"}
    if len(w) > 2 and w.endswith("y"):
        # quantify -> quantifies / quantified
        forms |= {w[:-1] + "ies", w[:-1] + "ied"}
    alts = "|".join(sorted(
        {re.escape(f) for f in forms},
        key=len, reverse=True,
    ))
    return re.compile(rf"\b(?:{alts})\b", re.IGNORECASE)

def _mask(text: str, words: list[str]) -> tuple[str, int]:
    hits = 0
    for w in sorted({x for x in words if x},
                    key=len, reverse=True):   # 长词先替换
        text, n = _word_pattern(w).subn("_____", text)
        hits += n
    return text, hits
```

为什么不能只加后缀

`mitigate + ing` 是错的，要去 e；`quantify + ed` 也错，要变 ied。漏掉这两条，原文里的变形就挖不到。

为什么按词长降序

短词先换会吃掉长词：`emit` 会把 `emission` 的前缀换掉，剩下残句，所以长词优先

`\b` 和 `subn`

`\b` 卡词边界，避免 `emit` 命中 `emitter` 中间；`subn` 顺手返回替换次数——就是「挖了几处」。

用到的知识

正则 alternation、词边界、`re.escape`、集合去重与排序、`subn` 的双返回值。

一句话：8 个词挖出 8 处，但有时 8 个词挖出 11 处——一个词可能在文中出现多次，那个数字就是 `hits` 累加出来的。

代码 09 • tools/memory.py:135–225 • 工具层

记忆要落成文件，复习日期要由代码算

```
for w in fail_words:
    _mark_fail(book[w])   # 间隔归 1 天、系数 -0.2、明天重来
for w in pass_words:
    _mark_pass(book[w])   # 系数 +0.05、1 天 → 3 天 → 上次 × 系数

schedule = {m["word"]: {
    "due": m["due"], "interval_days": m["interval"],
    "ease": m["ease"], "reps": m["reps"],
} for m in profile["mistakes"] if m["word"] in fail_words
                                  or m["word"] in pass_words}

return json.dumps({
    "schedule": schedule,      # 日期由代码算，模型只能抄
    "due_today": due_today,    # 今天该复习哪几个
    # 提示词里的档案是「本轮开始前」的快照，
    # 模型拿旧快照报次数会慢一格（实测报 4、实际 5）
    "counts": {m["word"]: int(m.get("count", 1))
               for m in profile["mistakes"]},
    "integration": "仅写入本地 JSON 档案；没有接入任何 App…",
    "reply_rule": "只能引用 due，不许自己说「七天后」…",
}, ensure_ascii=False)
```

什么是长期记忆

不是「模型记住聊天」，而是落成一份可读、可改、可版本管理的 JSON 。下次会话拼进提示词，它就知道你上次栽在哪。

为什么要排「下次哪天复习」

只记「错了 3 次」没用。同类软件真正的护城河是「这个词该在哪天重新出现」。所以每个词多四个字段：`interval / ease / reps / due`，递推在 `_mark_pass()` 里。

为什么日期和次数都要回传

提示词里的档案是本轮开始前的快照，模型拿它推算会慢一格；日期更是它最爱的自由发挥区，实测会顺口说「七天后」。**把算好的真值交给它，它只剩抄这一件事可做。**

用到的知识

JSON 持久化、日期运算（`timedelta`）、间隔重复（SM-2 的简化版）、幂等与计数、排序键（负数实现倒序）、快照与实时值

一句话：返回值不只是数据——它既**约束模型怎么说话**，也把「什么时候该复习」从模型的推测变成代码算出来的事实。

代码 10 • app.py:105–133 • 界面层

用生成器，让工具调用一步一步长出来

```python
for update in graph.stream(
    {"messages": lc_messages},
    config={"recursion_limit": RECURSION_LIMIT},
    stream_mode="updates",          # 每次只吐增量
):
    for node, payload in update.items():
        msgs = (payload or {}).get("messages") or []
        if node == "agent" and msgs:
            msg = msgs[-1]
            calls = getattr(msg, "tool_calls", None) or []
            for call in calls:
                args = json.dumps(call.get("args", {}),
                                  ensure_ascii=False)
                yield ("call", call.get("name"), args)
            if not calls and msg.content:
                final_text = msg.content
        elif node == "tools":
            for m in msgs:
                yield ("result", m.name, clip(m.content))

yield ("final", final_text or " (这一轮没有返回文字…) ")
```

为什么要这样写

商业 App 是黑盒推荐。这里要把每一步摊开，让「工具真的被调用」看得见，而不是模型凭空说话。

生成器的好处

函数一 `yield`，调用方就能立刻渲染，不用等全部跑完。12 次工具调用 = 12 次界面刷新。

`recursion_limit` 的坑

一次工具调用占 2 步（agent 一步、tools 一步）。主路径 12 次调用 = 24 步以上（实测 16～25 步），原来设 15，跑到第 7 次就抛 `GraphRecursionError`

用到的知识

生成器与 `yield`、迭代协议、`stream_mode` 增量 vs 全量、事件驱动 UI。

一句话：这个 bug 只在界面上出现，命令行测不出来——因为命令行测试用的是 limit=60。**换了入口就要重测。**

收束

十段代码，共用一条判断

按知识类型归类

语言基础	装饰器、生成器、类型注解、functools、pathlib
数据处理	JSON 持久化、CSV 编码（utf-8-sig）、正则与词边界
工程习惯	不可信输入、权限最小化、fail fast、单一数据源、缓存
Agent 层	状态图、Function Calling、上下文分层、按需加载、记忆

如果重做，我会改三处

① **词库换真实词典 API**　6161 条考试词表够撑住大部分，但派生词（retrieval / coherence）仍查不到，只能退回「模型推断 + 标注」。

② **工具参数用 JSON 而非竖线**　竖线省了模型输出，却引出了「六列挤位」那个 bug。

③ **把七个工具挂成 MCP server**　参数表和返回格式都不用改，只是换个暴露方式。

① 路径　② 循环　③ 上下文　④ 接口　⑤ 一致性　⑥ 校验　⑦ 容错　⑧ 算法　⑨ 记忆　⑩ 可见性

收尾那句：这十段里没有一段是「让模型更聪明」。它们都在做同一件事—— **把不该由模型决定的事，从模型手里拿走**：路径、等级、列名、次数、循环边界。